

(1) 包含关系。当可以从两个或两个以上的用例中提取公共行为时，应该使用包含关系来表示它们。其中这个提取出来的公共用例称为抽象用例，而把原始用例称为基本用例或基础用例。

(2) 扩展关系。如果一个用例明显地混合了两种或两种以上的不同场景，即根据情况可能发生多种分支，则可以将这个用例分为一个基本用例和一个或多个扩展用例，这样使描述可能更加清晰。

(3) 泛化关系。当多个用例共同拥有一种类似的结构和行为的时候，可以将它们的共性抽象成为父用例，其他的用例作为泛化关系中的子用例。在用例的泛化关系中，子用例是父用例的一种特殊形式，子用例继承了父用例所有的结构、行为和关系。

在实际工作中，系统分析师要谨慎选用这些关系。从上面的介绍可以看出，包含、扩展和泛化关系都会增加用例的个数，从而增加用例模型的复杂度。另外，一般都是在用例模型完成之后才对它进行调整，在用例模型建立之初不必急于抽象用例之间的关系。

11.5.2 分析模型

分析模型描述系统的基本逻辑结构，展示对象和类如何组成系统（静态模型），以及它们如何保持通信，实现系统行为（动态模型）。

建立分析模型的过程大致包括定义概念类、确定类之间的关系、为类添加职责、建立交互图等，其中有学者将前三个步骤统称为类-责任-协作者（Class-Responsibility-Collaborator, CRC）建模。

1. 定义概念类

OOA 的中心任务就是要找到系统中的对象或类，这些类将反映到系统设计中的软件类和系统实现中某个 OOP 语言声明的类。例如，在领域模型中，学员学习某门课程是一个事件，该事件记录了某个学员和某门课程在一定时期内的责任关系，表达的是领域概念；在系统设计模型中，学习记录就是一个软件类。虽然它们是不同的事物，但领域模型中的命名启发了后者的命名和定义，从而缩小了表示的差距。在整个系统开发过程中，要尽量使这些类或对象在不同阶段保持相同的名称。

发现类的方法有很多种，其中应用最广泛的是名词短语法。它的主要规则是先识别有关问题域文本描述中的名词或名词短语，然后将它们作为候选的概念类或属性，其具体步骤如下：

- (1) 阅读和理解需求文档或用例描述。
- (2) 筛选出名词或名词短语，建立初始类清单（候选类）。
- (3) 将候选类分成三类，分别是显而易见的类、明显无意义的类和不确定类别的类。
- (4) 舍弃明显无意义的类。
- (5) 小组讨论不确定类别的类，直到将它们都合并或调整到其他两个类别，并进行相应的操作。

例如，根据表 11-2 所描述的“开通课程”用例的事件流，可以获得候选概念类的清单，如表 11-3 所示。

表 11-3 候选概念类清单

名词类别	概念类列表
显而易见的类	课程、学习币、学员
明显无意义的类	请求、界面、信息
不确定类别的类	学员账户、课程类型、权限、用户名、费用、课程名称

经过简单分析可以得出，“学员账户”“用户名”“权限”可以归结到“学员”类，作为“学员”类的属性；“课程类型”和“课程名称”可以归结到“课程”类，作为“课程”类的属性；“费用”可以单独列为一个类，称为“费用清单”。这样，针对“开通课程”这个用例，就确定了4个类，分别是课程、学习币、学员和费用清单。

另外，也可以根据描述中的名词类别来发现候选类，例如，划分为人员、组织、物品、设备、事件、规格说明、业务规则或政策等。要注意的是，不是所有的名词或名词短语都是系统中的一个合适的候选类，因为有的在系统之外，有的与系统不相关，有的名词概念较小，只适合于作为某个候选类的属性。因此，必须对其进行一番筛选，把不合适的过滤掉。

2. 确定类之间的关系

当完成了类的寻找工作之后，就需要理清这些类之间的关系。类之间的主要关系有关联、依赖、泛化、聚合、组合和实现等，它们在UML中的表示方式如图11-10所示。

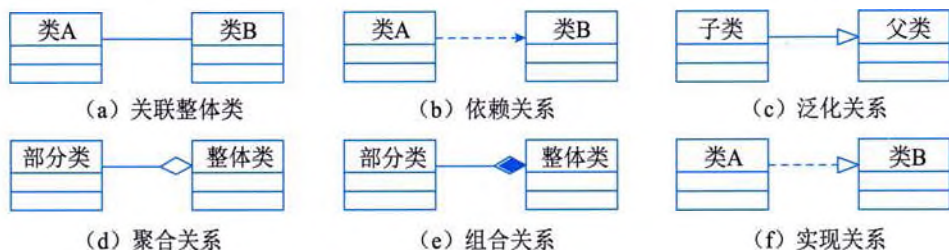


图 11-10 类之间的关系表示

(1) 关联关系。关联提供了不同类的对象之间的结构关系，它在一段时间内将多个类的实例连接在一起。关联体现的是对象实例之间的关系，而不表示两个类之间的关系。其余的关系涉及类元自身的描述，而不是它们的实例。对于关联关系的描述，可以使用关联名称、角色、多重性和导向性来说明，如图11-11所示。

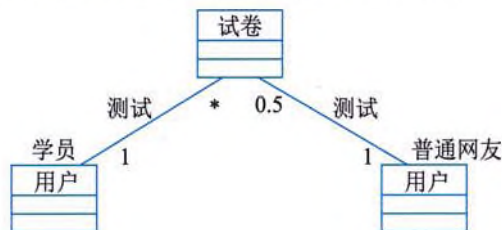


图 11-11 关联关系的表示

角色名称。多重性指定所在类可以实例化的对象数量（重数），即该类的多少个对象在一段特定

关联名称反映该关系的目，并且应该是一个动词，例如，图11-11中的“测试”。如果某种关联的含义对于开发人员和用户都是非常明确的，则可以省略关联名称。关联路径的两端为角色，角色规定了类在关联中所起的作用，例如，图11-11中的“学员”和“普通网友”。一般情况下，只有在关联名称不能明确表述时，才使用

的时间内可以与另一个类的一个对象相关联。例如，图 11-11 中的数字和“*”都表示关联，其中“*”等价于“0..*”。导向性表示可以通过关联从源类导向到目标类，也就是说，给定关联一端的对象就能够容易并直接地得到另一端的对象。导向性用一个箭头表示，如果没有箭头，就认为是一个双向关联或是一个未定义的关联。关联关系的表示方式，同样也适合其他关系的表示，只是箭线符号不同而已。

(2) 依赖关系。两个类 A 和 B，如果 B 的变化可能会引起 A 的变化，则称类 A 依赖于类 B。依赖可以由各种原因引起，例如，一个类向另一个类发送消息、一个类是另一个类的数据成员、一个类是另一个类的某个操作参数等。

(3) 泛化关系。泛化关系描述了一般事物与该事物中的特殊种类之间的关系，也就是父类与子类之间的关系。继承关系是泛化关系的反关系，也就是说，子类继承了父类，而父类则是子类的泛化。

(4) 共享聚集。共享聚集关系通常简称为聚合关系，它表示类之间的整体与部分的关系，其含义是“部分”可能同时属于多个“整体”，“部分”与“整体”的生命周期可以不相同。例如，汽车和车轮就是聚合关系，车子坏了，车轮还可以用；车轮坏了，可以再换一个。

(5) 组合聚集。组合聚集关系通常简称为组合关系，它也是表示类之间的整体与部分的关系。与聚合关系的区别在于，组合关系中的“部分”只能属于一个“整体”，“部分”与“整体”的生命周期相同，“部分”随着“整体”的创建而创建，也随着“整体”的消亡而消亡。例如，一个公司包含多个部门，它们之间的关系就是组合关系。公司一旦倒闭，也就无所谓部门了。

(6) 实现关系。实现关系将说明和实现联系起来。接口是对行为而非实现的说明，而类中则包含了实现的结构。一个或多个类可以实现一个接口，而每个类分别实现接口中的操作。

确定了类之间的关系之后，通过 UML 的类图将这些关系记录下来，形成领域模型。例如，图 11-12 就是与在线教育平台系统相关的一个简单的领域模型。

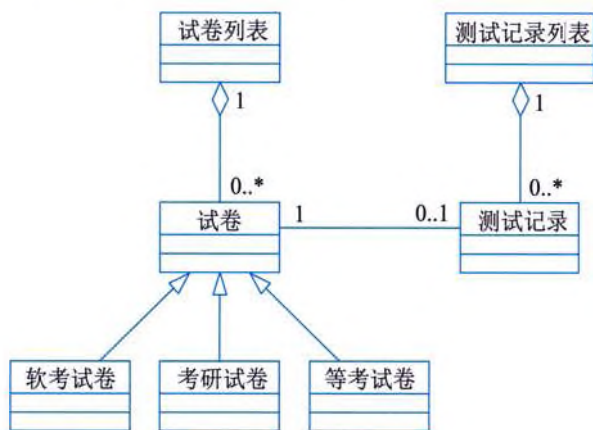


图 11-12 领域模型示例

3. 为类添加职责

找到了反映问题域本质的主要概念类，而且理清了它们之间的协作关系之后，系统分析师

就可以为这些类添加其相应的职责。类的职责包括两个方面的内容，一个是类所维护的知识，即成员变量或属性；另一个是类能够执行的行为，即成员方法或责任。

属性是描述类静态特征的一个数据项。系统分析师可以与用户进行交谈，提出问题来帮助寻找类的属性。从概念建模的角度来看，属性越简单越好。要保持属性的简单性，应该做到只定义与系统责任和系统目标有关的属性；使用简单数据类型来定义属性，不使用可由其他属性导出的属性（冗余属性）；不为类关联定义属性。最后，要对属性加以说明，包括名称、解释和数据类型，以及其他的一些要求。

对于类的责任的确定，可以根据用例描述中的动词来进行判断，然后再进行筛选。这个过程与类的识别过程是类似的，此处不再赘述。系统分析师应该通用性地描述类的成员方法，例如，“交费”和“组卷”等。另外，根据封装性原则，信息和与其相关的行为应该存在于同一个类中。关于一个事物的信息应该包含在单个类中，而不是分布在多个类中。但是，在适当的时候，可以在相关的类之间分享责任。

要注意的是，为类添加职责与找到类之间的关系一样，这个阶段也只能找到那些主要的、明显的、与业务规则相关的部分。切忌在这个阶段不断地细化，甚至引入一些与具体实现相关的技术内容（例如，数据库和分布式对象等）。

4. 建立交互图

多个对象的行为通常采用对象交互来表示，UML 2.0 提供的交互图有顺序图、交互概览图、通信图和定时图。每种图出于不同视点对行为有不同的表现能力，其中最常用的是顺序图，几乎可以用在任何系统的场合。顺序图的基本元素有对象、参与者、生命线、激活框、消息和消息路线，其中消息是顺序图的灵魂。当对象行为复杂并存在多种不同状态转换时，还要用到反映对象状态变化的状态图。

5. 分析模型的详细程度问题

对于分析模型的详细程度，各种文献说法不一，在实践中的做法也不一样。有些文献建议只列出类以及类之间的主要关系，不要对关系进行描述，更不要体现类的职责；而有些文献则认为应该将这些东西都列出来。其实，干巴巴地讨论这个问题是没有任何意义的。在整个系统开发的过程中，分析模型是不断演变的，应随着开发的深入加入新的内容，或修改已有内容，逐渐完善，演变完整。最初的分析模型主要是围绕着领域知识进行的，对现实的事物进行建模。而后，则不断地加入设计的元素，最终演变成为运行于计算机上的系统。

总之，模型不是要开发的目标产物，而是开发过程中的一个辅助工作，只要能够利用其帮助团队更好地开发，详细也罢，简约也罢，都是好模型。

11.6 需求定义

系统分析师在获取了用户的需求，并进行了详细分析之后，接下来的工作就需要把这些软件需求形成文档，作为系统后续开发的基础，这就是需求定义。